A Speaker-Safe Pipeline for Detecting Parkinson's Disease from Speech

Arannamoy Mondal

11 October 2026

Abstract

This note explains the full pipeline in plain English. The question is whether an elderly voice sounds like Parkinson's disease. Young controls stay on disk and are not labeled. The published score is computed on speakers, not on short windows. A flowchart shows the whole path. Tables then state what each of the one hundred steps does, and how you know that step is finished.

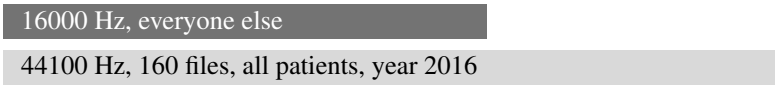
1 The question

Three groups sit in the corpus folder. Only two enter the experiment.

Group	People	Files	Role
Parkinson's disease	28 folders, 25 people	437	Label 1
Elderly healthy control	22	349	Label 0
Young healthy control	15	45	Not used

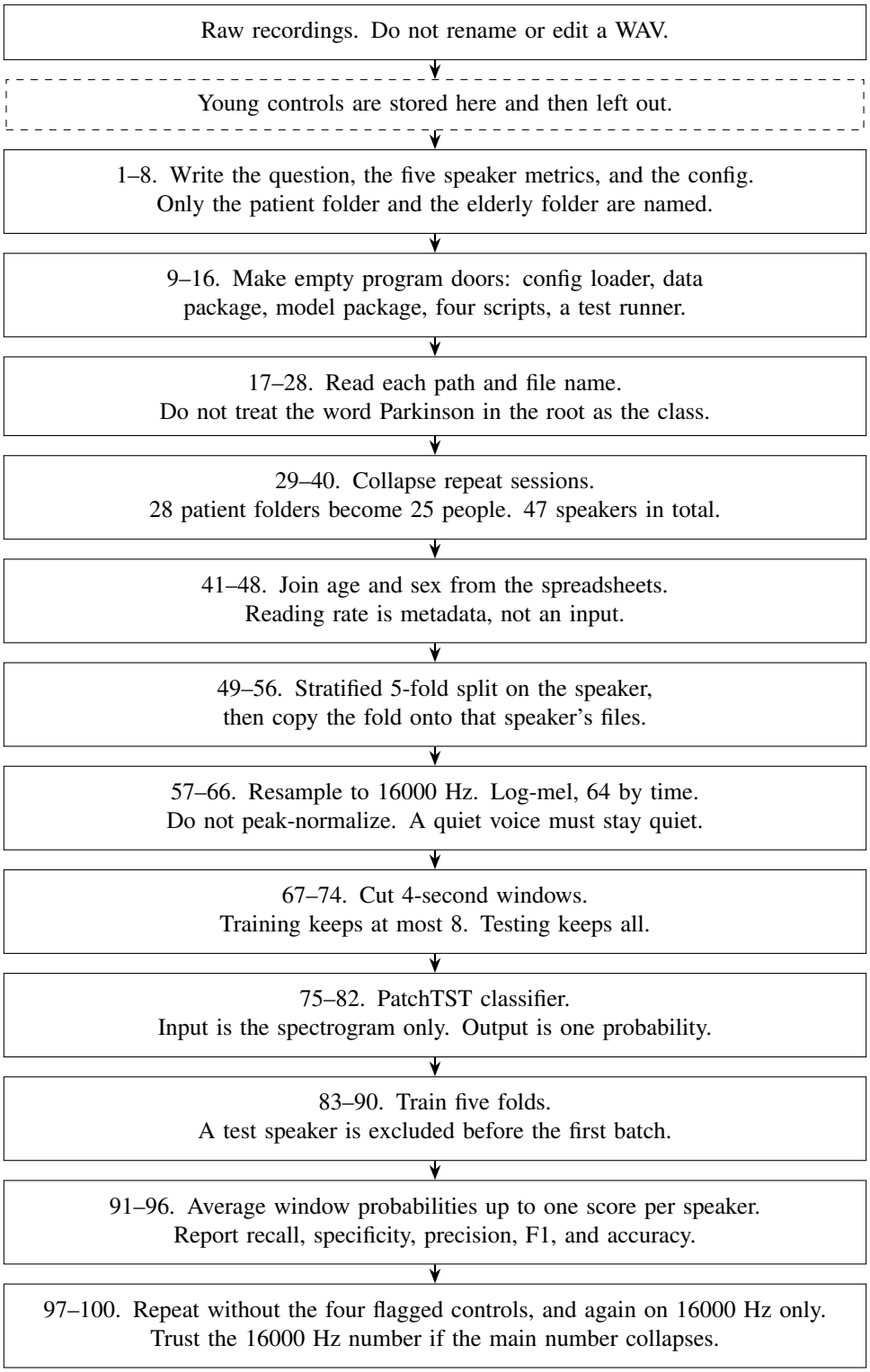
The young speakers are 19 to 29 years old and recorded only two readings plus a word list. The patients and the elderly controls are about 67 and recorded the same full protocol. Mixing the young group into the healthy class lets a model detect age and call it disease.

About 160 patient files from 2016 are stored at 44100 Hz. Every elderly file is stored at 16000 Hz. If that difference is left in the features, the model can detect the microphone. Every file is resampled to 16000 Hz before a feature is computed. A later run drops the 44100 Hz sessions and checks whether the score survives.



2 The full pipeline

Read this figure from top to bottom. A later box is not allowed to start until the box above it is true.



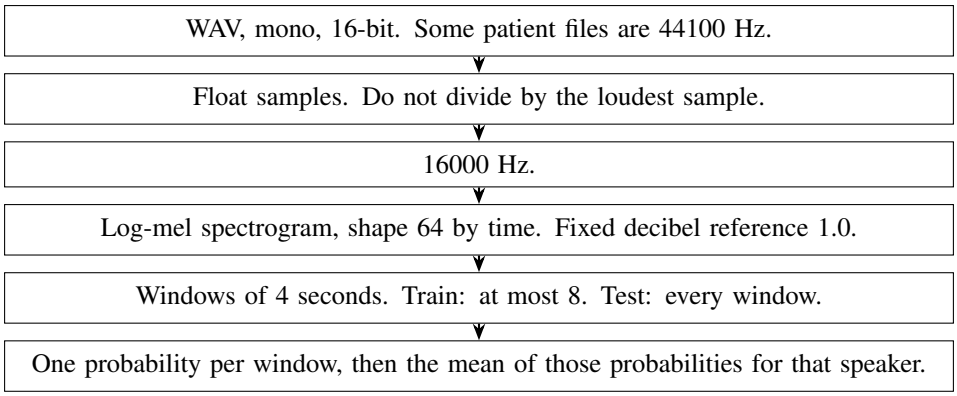
3 What one recording becomes

A 2017 file name has four parts. The date is not part of the person code.

B1	APGANRET55F	17032017	1104
task	person code	17/03/2017	11:04

A 2016 name is shorter at the end: six date digits, not eight. `260516` must be stored as 26/05/2016, not as the year 16.

After the name is read, the sound follows one path.



4 The five numbers

Each number answers a different question. They are computed only after the windows of one speaker have been averaged.

Name	Formula	What it answers
Recall	TP / (TP + FN)	Of the patients, how many were caught?
Specificity	TN / (TN + FP)	Of the elderly controls, how many were left alone?
Precision	TP / (TP + FP)	Of the people called ill, how many were ill in this sample?
F1	harmonic mean of precision and recall	Did we buy one of those by destroying the other?
Accuracy	(TP + TN) / all speakers	How often was the speaker call right?

Recall and specificity are the clinical pair. F1 is the comparison number. Accuracy is reported, not trusted alone. Precision here is not the positive predictive value of a clinic, because this sample is about half patients and the community is not.

5 Steps 1 to 8: freeze the decision

Step	What this step does	How you know it is done
1	Write the question in one sentence: Parkinson's disease versus elderly healthy control, from speech.	The sentence does not mention the young group.
2	Name the published score: speaker-level precision, recall, specificity, accuracy, and F1.	Window accuracy is listed only as a debug number.
3	Record three citations: VoxTester 2016, IEEE Access 2017, DataPort DOI 10.21227/aw6b-tg17.	They are in the README, not only in a notebook.
4	Point the config at the two group folders.	The young folder is absent from `groups`.
5	Set rate 16000, tasks all, 5 folds, seed 42, group by speaker.	No other file contains these numbers.
6	Ignore generated data, run outputs, the virtual environment, the corpus, and the zip.	A status listing does not show a WAV.
7	Leave the WAV tree where it is.	No script writes inside it.
8	Move the notebook to `notebooks/`.	The notebook is marked plots only.

The phase is finished when the README states the cohort and the metric, and the config names exactly two groups.

6 Steps 9 to 16: empty doors

Step	What this step does	How you know it is done
9	`config.py` loads the yaml.	A missing key is reported. A missing value is not invented.
10	Add the empty data package file.	`import patchtst.data` works.
11	Add the empty model package file.	The file is empty. There is no network yet.
12	Add the manifest script.	It loads the config and exits. It does not parse names.
13	Add the feature-cache script.	It loads the config and exits. It does not read audio.
14	Add `scripts/train.py`.	It loads the config and exits. It does not build a model.
15	Add `scripts/evaluate.py`.	It loads the config and exits. It does not score anyone.
16	Add a test runner and one test that fails on purpose.	The runner prints that failure. A missing test is not a pass.

These four scripts look the same because their real jobs are not allowed yet. The file name reserves the door.

7 Steps 17 to 28: read paths and names

Step	What this step does	How you know it is done
17	Match `Elderly Healthy Control` and `People with Parkinson`.	The function does not search for the word Parkinson alone.
18	Check an elderly path.	The label is 0.
19	Check a patient path.	The label is 1.
20	Check the dataset root.	It is refused. It is never labeled PD.
21	Parse a 2017 name: eight date digits plus four time digits.	The date and the token are separate strings.
22	Parse a 2016 name: six date digits plus four time digits.	The year is 2016, not 16.
23	Map a leading `PR11` to `PR1`.	That file is not a new task. A normal PR1 file is unchanged.
24	Remove a trailing double dot before `.wav`.	Alberto R's word-list file parses as 10/01/2017.
25	Delete spaces inside the person code.	`VDIET AS47M` becomes `VDIETAS47M`.
26	Read the task from the start of the name, longer codes first.	`FB1` is not read as `B1`.
27	Walk only the two configured folders.	Young files never enter the table.
28	Read rate, channels, format, and duration from the header.	The waveform is not loaded just to index the file.

The phase is finished at 786 rows: 437 patient files and 349 elderly files. Every file is mono 16-bit PCM. About 160 patient files are 44100 Hz. Every elderly file is 16000 Hz.

8 Steps 29 to 40: one person, one identity

Step	What this step does	How you know it is done
29	Replace a stray person code with the code used by the rest of that folder.	Antonia G's `63F` file becomes `52F` and is flagged.
30	Count that flag.	Exactly one file is flagged.
31	Join folders with the same group and the same person code.	`VSIPTIOZ46M` is one person.
32	Join folders with the same name and the same birth year and sex, even if the code spelling differs.	Nicola S and Roberto R each collapse.
33	Do not join across groups.	An elderly code cannot merge with a patient code.

Step	What this step does	How you know it is done
34	Build the speaker id from the group plus the surviving code.	The id does not contain the folder path.
35	Count speakers.	47 speakers: 25 patients, 22 elderly.
36	Check the three repeat people.	Each has two folders and both sample rates.
37	Check the other Vito S.	The code ending `47M` is a different speaker id.
38	Store native rate, duration, task, and session date on every row.	Later code does not parse the file name again.
39	Write `data/manifest.csv` and `data/speakers.csv`.	A second run does not change the row count.
40	Print the three linked speakers.	The merge is visible without opening the table.

9 Steps 41 to 48: join the spreadsheets

Step	What this step does	How you know it is done
41	Read `TAB 5.xlsx` and `Tab 3.xlsx`, not the code legend.	Sex and age columns are found by name.
42	Require sheet sex to equal the sex letter in the file name.	A mismatch stops the build.
43	Match swapped elderly names: VITO L, LISCO G, SUMMO L.	All 22 elderly folders match.
44	Match folder Mario B to the only Mario row.	He is not left unmatched. Age is within one year.
45	Store sheet age and also session year minus birth year.	The two ages differ by at most 1 for every speaker.
46	Flag Agnese P, Gilda C, Giovanna G, and Vitantonio D.	Only those four have `sheet_flag`.
47	Keep reading-rate columns as metadata.	No training function accepts them as input.
48	Stop if any speaker has no sheet row.	A missing age is not filled with an empty value.

The phase is finished when 47 of 47 speakers match and there are zero sex mismatches.

10 Steps 49 to 56: split by speaker

Step	What this step does	How you know it is done
49	Build folds from one row per speaker, not per file.	A person with 16 files is one unit.
50	Use a stratified 5-fold split, shuffled, seed 42.	A plain group split on a patient-then-control sort is not used.
51	Copy the fold onto every file of that speaker.	The column is the test fold.
52	Check that no speaker appears in two folds.	A forgotten merge fails this check.
53	Check that every fold contains both labels.	Each test fold has about 5 patients and 4 or 5 controls.
54	Write `data/folds.csv`.	It has 47 rows.
55	Add a switch to drop the four flagged controls.	The switch is off in the main run.
56	Add a switch to keep only native 16000 Hz files.	This is the microphone run, not the main run.

11 Steps 57 to 66: make the spectrogram

Step	What this step does	How you know it is done
57	Read samples as floating point on the file's own scale.	The audio is not divided by its loudest sample.
58	Resample to 16000 Hz only when the stored rate differs.	A 16000 Hz file is not resampled twice.
59	Check one 44100 Hz file.	Mel frames are about duration times 100, not times 441.
60	Compute log-mel: 64 bands, Fourier size 512, window 400, hop 160.	The shape is 64 by time.
61	Use a fixed decibel reference of 1.0.	A quiet file stays quieter than a loud file.
62	Name each cache file by a hash of the WAV path.	Changing the task list cannot attach the wrong array.
63	Reuse a cache file only if its first dimension is 64.	A changed band count recomputes it.
64	Write the cache path back into the manifest.	786 paths exist.
65	Keep feature settings in the config, not in the audio code.	Changing the hop means editing the yaml only.
66	Time one full cache and write the seconds down.	You know whether training must read the cache.

A 73-second file that began at 44100 Hz has about 7300 frames. A 6-second syllable file has about 600.

12 Steps 67 to 74: cut windows

Step	What this step does	How you know it is done
67	Define one window as 4.0 seconds, 400 frames, hop 2.0 seconds.	Both numbers come from the config.
68	Pad a short clip by repeating the edge.	The shortest vowel still yields one window.
69	In training, keep at most 8 windows, evenly spaced.	A 250-second reading does not yield 100 windows.
70	In testing, keep every window.	The cap is not applied in evaluation.
71	Copy the file label and the speaker id onto each window.	The dataset never receives a folder name.
72	Check the batch shape.	It is count, then 64, then 400.
73	Refuse a batch that mixes two folds.	The dataset is built from one split.
74	Let the config choose all tasks or a shorter list.	Vowels only does not rebuild speaker ids.

Do not start the network before this check: one long patient file contributes 8 training windows and more than 8 test windows.

13 Steps 75 to 82: the classifier

Step	What this step does	How you know it is done
75	Add the training library.	The data package still imports without it.
76	Cut time into patches. Length and stride come from the config.	The 64 channels stay independent.
77	Put a classification head on the encoder.	The output is one logit, not a forecast.
78	Write the input contract in words.	A batch of 64 by 400 goes in. One probability comes out.
79	Run one dummy batch on the CPU.	The output shape is batch size by 1.
80	Save the patch length beside the checkpoint.	It is not guessed later from the weight file.
81	Accept only the spectrogram.	Sex, age, reading rate, and native rate are not inputs.
82	Count the parameters and write the number down.	The model is not larger than 47 speakers can support.

14 Steps 83 to 90: train

Step	What this step does	How you know it is done
83	Train one fold.	Test speakers are excluded before the first batch.
84	Write the run to its own dated folder, with the fold and a config hash.	A second run does not overwrite the first.
85	Copy the yaml into that folder.	The copy is the config that was used, not a later edit.
86	Save the checkpoint there.	The path is in a small run file.
87	Write the loss for each epoch in that folder.	The notebook is not the log.
88	Balance the loss.	437 patient files do not silently outweigh 349 elderly files.
89	Stop early on a validation speaker loss.	Early stopping cannot see the test fold.
90	Repeat for folds 0 through 4 with the same seed.	Five run folders exist.

Opening any run folder must show the fold, the seed, and the config hash without reading the source code.

15 Steps 91 to 100: the score, and whether it is real

Step	What this step does	How you know it is done
91	Score every test window.	The window table has speaker id, path, task, and probability.
92	Average probabilities by file, then by speaker.	The speaker score table exists.
93	Compute the five metrics on speaker scores.	The metrics file exists. The threshold was not chosen on the test fold.
94	Pool the five test folds and compute the five numbers once.	This pooled number is the one you quote.
95	Also report the mean and the spread across folds.	The best fold is not quoted.
96	Compare the speaker count in the score table with the test speakers.	A dropped speaker fails the run.
97	Repeat evaluation without the four flagged controls.	You record whether the pooled F1 moves.
98	Retrain and evaluate on native 16000 Hz only.	The seven patients who exist only at 44100 Hz are gone.

Step	What this step does	How you know it is done
99	If that F1 collapses, write it in the run note.	The mixed-rate number is not published as the disease result.
100	Plot from the speaker score table only.	The notebook does not open a WAV except through that table.

Three numbers then sit side by side: the main pooled F1, the F1 without the four flagged controls, and the F1 on 16000 Hz only. The third is the number that is trusted if the first was helped by the old microphone.

References

[1] G. Dimauro, D. Caivano, V. Bevilacqua, F. Girardi, and V. Napoletano, “VoxTester, software for digital evaluation of speech changes in Parkinson disease,” IEEE MeMeA, 2016.

[2] G. Dimauro, V. Di Nicola, V. Bevilacqua, D. Caivano, and F. Girardi, “Assessment of Speech Intelligibility in Parkinson’s Disease Using a Speech-To-Text System,” IEEE Access, 2017.

[3] G. Dimauro and F. Girardi, “Italian Parkinson’s Voice and Speech,” IEEE DataPort, 2019. DOI 10.21227/aw6b-tg17.